



ITESO

Universidad Jesuita
de Guadalajara

Propuesta del proyecto final

Materia: Infraestructura para el desarrollo continuo

Alumno: Federico Humberto Zaragoza Manuel

Profesor: Luis Eduardo Estrada Rodríguez

Tlaquepaque, Jal., 04 de octubre de 2026

Contenido

Título.....	3
Descripción del proyecto.....	3
Problemática y justificación	4
Componentes y funcionalidades	6
Diagrama UML	10
Stack Tecnológico.....	11
Diagrama de infraestructura	13
Plan de trabajo.....	15
Pruebas	15
Docker Hub y Estrategia de Contenedores	21
GitHub Project Board y plan de trabajo	22

Titulo

PLANORA: Plataforma Integral de Gestión Operativa, Gastronomic y Financiera para Organizadores Profesionales de Eventos.

El nombre **PLANORA** surge de la fusión conceptual de *Planning* (planificación metódica, orden y logística rigurosa) y *Aura* (la atmósfera, distinción y esencia única que define a cada evento).

Es una marca corta (3 sílabas), fonéticamente armónica, memorable y con proyección internacional. Al evitar términos exclusivamente nupciales (como *wedding* o *vow*), PLANORA permite abarcar holísticamente:

- Bodas y celebraciones de XV años.
- Fiestas de aniversario y graduaciones.
- Galas sociales y convenciones corporativas.

Descripción del proyecto

PLANORA es una plataforma web *Software as a Service* (SaaS) orientada al segmento B2B, concebida como el centro de control operativo, logístico y financiero para organizadores profesionales de eventos independientes y agencias (*wedding planners*, coordinadores corporativos y sociales).

El sistema responde a una necesidad crítica de la industria: los coordinadores administran proyectos de alto impacto emocional y financiero donde un solo error en el conteo de invitados, un sobrecupo en las mesas, una confusión con platillos especiales o una alergia alimentaria no notificada a tiempo al equipo de cocina puede arruinar una celebración o desencadenar emergencias médicas y pérdidas económicas.

PLANORA no es un portal de descubrimiento de eventos ni una boletería de venta masiva de entradas al estilo de Eventbrite. Es una herramienta de gestión interna del organizador diseñada para:

1. Administrar el ciclo de vida completo de múltiples eventos de forma simultánea.
2. Mantener un control exacto de la lista de comensales, confirmaciones de asistencia (RSVP) y distribución física de mesas.
3. Centralizar las preferencias gastronómicas y emitir el Reporte Oficial para Catering con alertas de alergias para evitar contaminación cruzada en cocina.
4. Monitorear el presupuesto inicial frente a los gastos reales mediante alertas de semáforo preventivas (Verde, Amarillo, Rojo).
5. Centralizar el directorio de proveedores, contratos y comprobantes de pago en la nube.
6. Coordinar el cronograma del "minuto a minuto" del día del evento garantizando que no existan empalmes de horarios entre proveedores.

Problemática y justificación

1.- Problemática identificada:

Como parte de una exploración inicial con una persona con experiencia práctica en la coordinación y producción de eventos, se identificaron los siguientes puntos de dolor operativos:

1. Gestión de Invitados y Conteo de Asistentes:

En eventos sociales resulta muy complejo llevar el conteo exacto de personas confirmadas, declinadas y acompañantes (+1s). Cualquier error deriva en sobrecostos por cubiertos no utilizados o falta de asientos el día del evento.

2. Distribución y Aforo de Mesas:

Asignar manualmente a decenas o cientos de invitados en mesas respetando su capacidad física máxima y afinidades familiares/sociales es una tarea laboriosa propensa a sobrecupos de último minuto.

3. Alimentación y Asignación de Menús:

Es complicado determinar con exactitud qué platillo corresponde a cada persona una vez sentada en su mesa (menú regular, menú infantil, vegetariano, vegano, kosher), provocando confusiones y demoras en el servicio de los meseros.

4. Alergias y Restricciones Médicas Críticas:

Representa un punto de alto riesgo para la seguridad física de los invitados. No contar con un registro riguroso de alergias severas (mariscos, frutos secos, celiaquía/gluten, lactosa) puede desencadenar emergencias médicas si la cocina no recibe indicaciones precisas por mesa.

5. Gestión de Bebidas:

Coordinar insumos y cantidades de bebidas (barra libre, destilados por mesa, descorche, bebidas sin alcohol) suele generar dudas en las compras o desabasto durante las horas pico de la fiesta.

6. Coordinación de Transporte:

Cuando el organizador asume la logística de traslados (autobuses, camionetas para comitivas, traslados entre templo y salón), coordinar horarios, capacidades de vehículos y puntos de reunión incrementa considerablemente la carga de trabajo.

7. Dispersión de la Información (Hipótesis Inicial de Trabajo):

Se plantea como hipótesis que los organizadores gestionan actualmente estos procesos recurriendo a múltiples herramientas separadas (hojas de cálculo en Excel, conversaciones y confirmaciones informales por WhatsApp, y notas o contratos en Google Drive), lo que provoca información desactualizada y falta de trazabilidad.

Componentes y funcionalidades

El sistema está compuesto por 10 módulos altamente cohesivos:

1. UserComponent (Usuarios y Autenticación)

- Propósito: Identidad, seguridad y control de acceso basado en roles (admin, organizer, client).
- CRUD: createUser(), getUserById(), updateUser(), deleteUser().
- Métodos de Negocio: authenticate() (firma JWT con Web Crypto API), changePassword(), deactivateUser().

2. ClientComponent (Clientes y Anfitriones)

- Propósito: Administración de anfitriones o empresas contratantes del evento.
- CRUD: createClient(), getClientById(), getClientsByOrganizer(), updateClient(), deleteClient().
- Métodos de Negocio: linkUserAccount() (vincula al portal del cliente), getClientHistorySummary().

3. EventComponent (Eventos)

- Propósito: Agregado raíz del sistema. Orquesta estados, fechas, presupuestos y progreso general.
- CRUD: createEvent(), getEventById(), listEvents(), updateEvent(), deleteEvent().
- Métodos de Negocio:
 - confirmEvent(): Valida fecha futura, cliente asociado y presupuesto base antes de pasar a confirmed.
 - cancelEvent(): Desactiva tareas pendientes en cascada lógica y congela el cronograma.
 - calculateEventProgress(): Retorna porcentaje de avance ponderado (0% – 100%) basado en tareas terminadas.

- `cloneEventStructure()`: Permite clonar cronograma y tareas de un evento exitoso previo.

4. GuestComponent (Invitados, Mesas y Gastronomía)

- Propósito: Control de comensales, RSVP, mesas, asignación de menús y prevención de riesgos por alergias.
- CRUD: `addGuest()`, `getGuestById()`, `listGuestsByEvent()`, `updateGuest()`, `removeGuest()`.
- Métodos de Negocio:
 - `registerRsvp(guestId, status, plusOnes)`: Actualiza confirmación y valida límite de acompañantes.
 - `assignTable(guestId, tableNumber, maxCapacity)`: Valida algorítmicamente que los comensales no excedan el aforo máximo de la mesa física.
 - `registerDietaryProfile(guestId, mealPref, allergies, notes)`: Sella tipo de menú (standard, vegetarian, vegan, child, kosher, other) y alérgenos médicos.
 - `generateCateringDietaryReport(eventId)`: Genera el consolidado para cocina: desglose de platillos por mesa y alertas de alergias geolocalizadas por mesa.
 - `calculateRsvpMetrics()`: Métricas de confirmados, declinados y porcentaje de asistencia.

5. VendorComponent (Proveedores)

- Propósito: Directorio y evaluación de empresas externas (catering, música, transporte, fotografía).
- CRUD: `createVendor()`, `getVendorById()`, `listVendors()`, `updateVendor()`, `deleteVendor()`.
- Métodos de Negocio: `updateRating()` (calificación promedio de desempeño en escala 1.0 a 5.0).

6. EventServiceComponent (Contrataciones)

- Propósito: Formalización de contratos entre eventos y servicios de proveedores.
- CRUD: `contractService()`, `getContractById()`, `listContractsByEvent()`, `cancelContract()`.
- Métodos de Negocio: `attachContractDocument()` (archivo PDF en Cloudflare R2), `generateExpense()` (crea gasto presupuestario automático en estado pending).

7. BudgetComponent / ExpenseComponent (Presupuesto y Gastos)

- Propósito: Control financiero, desembolsos y conciliación presupuestaria.
- CRUD: `recordExpense()`, `getExpenseById()`, `listExpensesByEvent()`, `updateExpense()`, `deleteExpense()`.
- Métodos de Negocio:
 - `calculateBudgetStatus()`: Computa $\text{Balance} = \text{Presupuesto Inicial} - \sum \text{Gastos Registrados}$.
 - `checkBudgetOverrunAlert()`: Emite nivel de semáforo: Verde ($< 90\%$), Amarillo ($90\% - 99\%$) o Rojo ($\geq 100\%$).
 - `registerPayment()`: Sella fecha `paid_at`, transiciona a paid y vincula el recibo en R2.

8. TaskComponent (Tareas Operativas)

- Propósito: Lista de actividades con responsables, fechas límite y prioridades.
- CRUD: `createTask()`, `getTaskById()`, `listTasksByEvent()`, `updateTask()`, `deleteTask()`.
- Métodos de Negocio: `completeTask()` (marca finalizada y actualiza avance del evento), `getOverdueTasks()`.

9. AgendaComponent (Cronograma Minuto a Minuto)

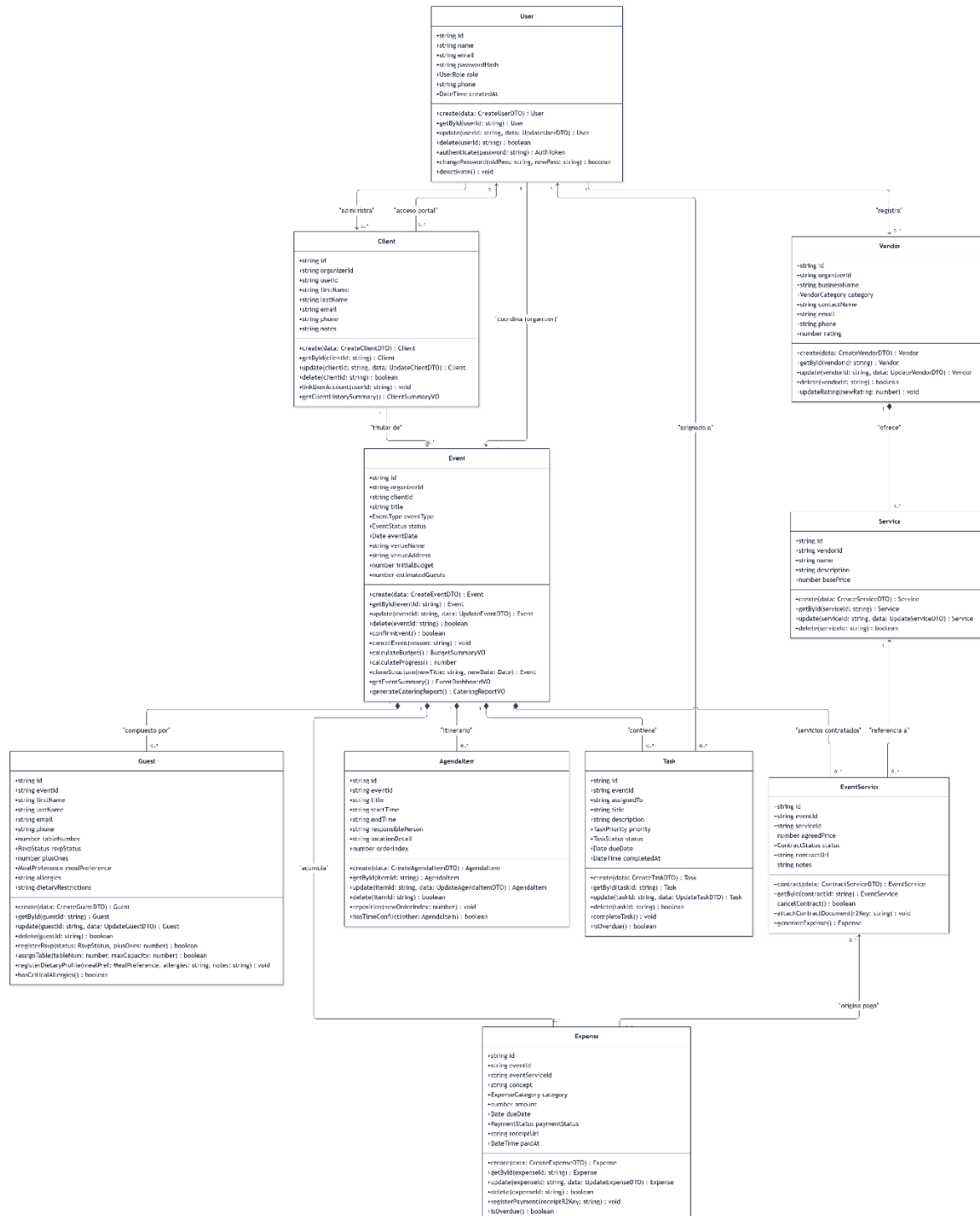
- Propósito: Coordinación del itinerario horario del día del evento.

- CRUD: addAgendaItem(), getAgendaItemById(), listAgendaByEvent(), updateAgendaItem(), deleteAgendaItem().
- Métodos de Negocio:
 - reorderTimeline(): Recalcula y persiste el orden secuencial (order_index).
 - detectTimeConflicts(): Detecta solapamientos horarios en la misma locación o con el mismo responsable.

10. DashboardComponent (Dashboard Ejecutivo)

- Propósito: Métricas agregadas y analítica para el organizador.
- Métodos de Negocio: getOrganizerOverview() (eventos activos, monto total presupuestado y tareas atrasadas).

Diagrama UML



Stack Tecnológico

Tecnología	Rol	¿Qué parte del proyecto resuelve?
React 18 + TypeScript + Vite	Frontend SPA	Interfaz de usuario dinámica para gestión de mesas, comensales y cronogramas.
Tailwind CSS	Framework de Estilos	Diseño profesional y consistente del dashboard con soporte móvil.
Hono	Backend API Framework	Enrutamiento de la API, validación con Zod y middleware JWT.
Cloudflare Workers	Cómputo Serverless Backend	Ejecuta la API sin necesidad de servidores o clústeres permanentes.
Cloudflare D1	Base de Datos Relacional	Almacenamiento de usuarios, eventos, comensales y gastos.
Drizzle ORM	Mapeo Objeto-Relacional	Consultas seguras y gestión de migraciones para D1.
Cloudflare R2	Almacenamiento de Objetos	Almacenamiento de contratos PDF y fotografías de comprobantes de pago.
Cloudflare KV	Almacén Clave-Valor	Lista negra de tokens revocados y control de cuotas.
Vitest	Framework de Testing	Pruebas unitarias de cálculo presupuestario y aforo de mesas.
Git & GitHub	Control de Versiones	Trazabilidad del desarrollo, revisión de Pull Requests y gestión de ramas.

Tecnología	Rol	¿Qué parte del proyecto resuelve?
GitHub Actions	Automatización CI/CD	Ejecución automatizada de pruebas, linting, compilación Docker y despliegues.
Docker & Docker Hub	Contenedores y Registro OCI	Entorno de build y testing reproducible, además de distribución de imágenes.

Diagrama de infraestructura

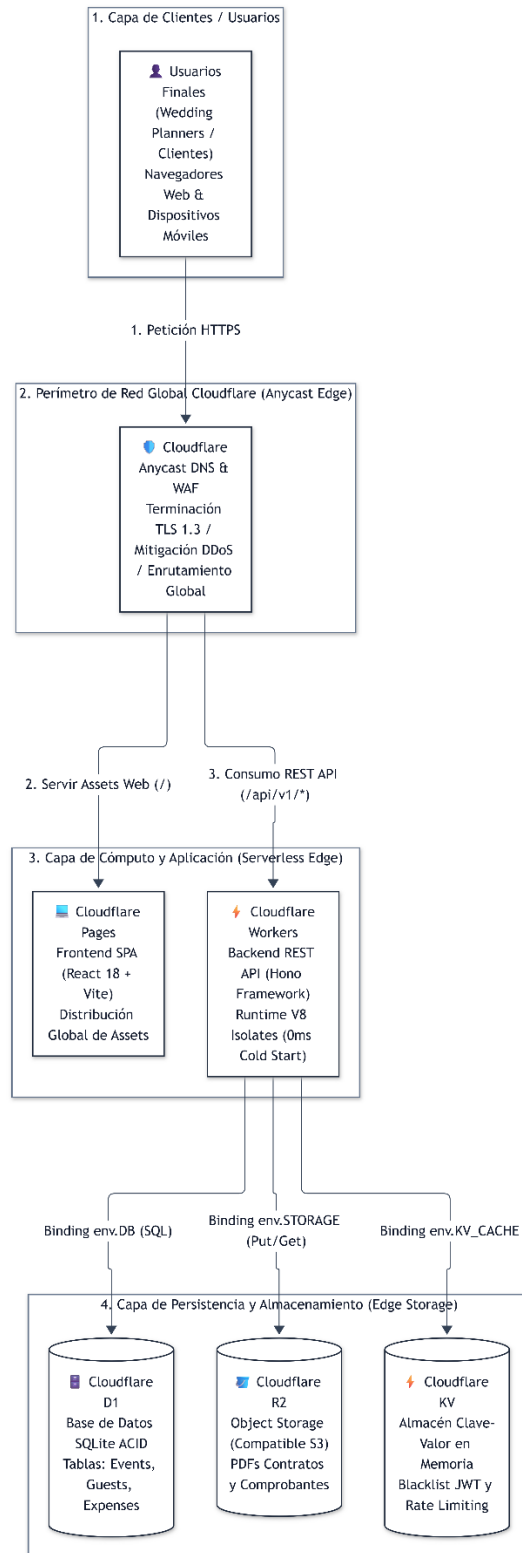
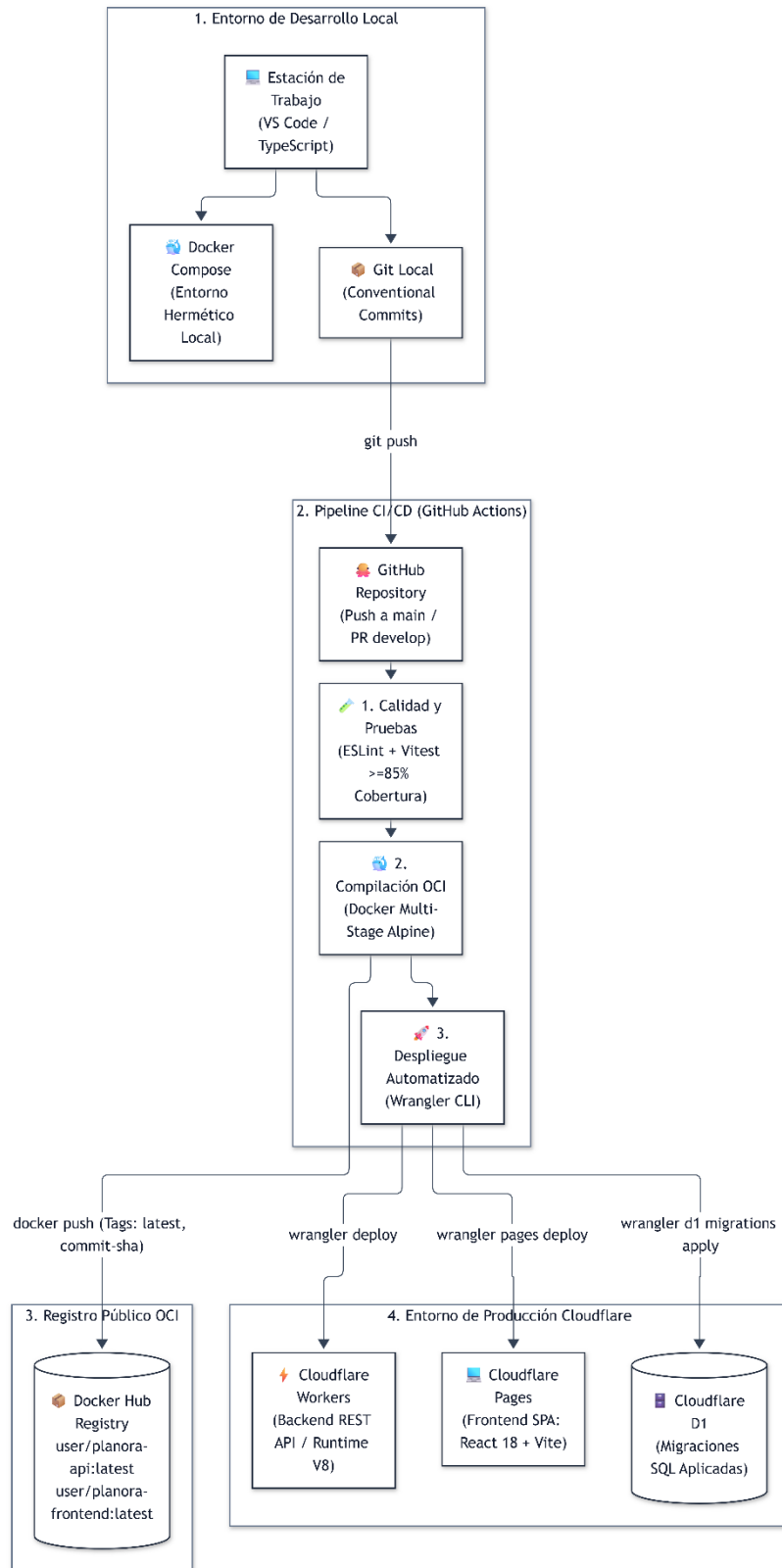
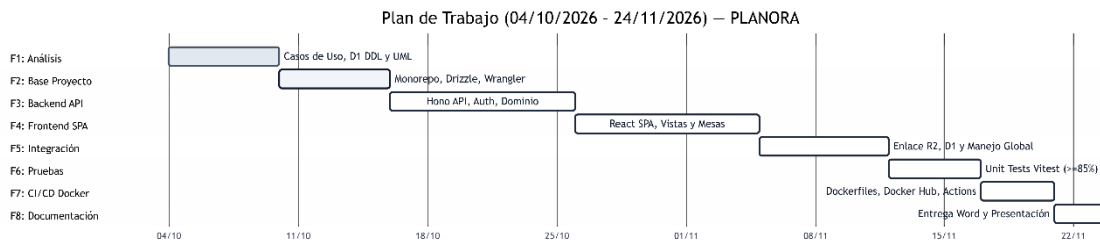


Diagrama de Deployment y Flujo de Integración Continua (CI/CD)



Plan de trabajo



Pruebas

Caso de Prueba 1: Conciliación Financiera y Semáforo de Presupuesto (CP-FIN-01)

- **Módulo / Componente:** BudgetCalculator / ExpenseService (Módulo de Presupuesto y Gastos).
- **Propósito:** Validar que el motor financiero compute con precisión el balance remanente, el porcentaje ejecutado de inversión y la activación del semáforo preventivo de sobre costo conforme a los umbrales de negocio establecidos.
- **Precondiciones:** Evento registrado con un presupuesto base pactado con el cliente.
- **Escenarios de Prueba Evaluados:**
 1. **Escenario 1.1 — Margen Seguro (Semáforo Verde):**

Entrada: Presupuesto inicial de \$100,000 MXN; gastos registrados y confirmados por \$45,000 MXN.

Ejecución: El servicio calcula el balance remanente y el porcentaje ejecutado (\$55,000 MXN restantes, 45.0% consumido).

Resultado Esperado: Estado del semáforo en verde y bandera de sobre costo inactiva (isOverrun = false).
 2. **Escenario 1.2 — Alerta Preventiva (Semáforo Amarillo):**

Entrada: Presupuesto inicial de \$100,000 MXN; gastos registrados acumulados por \$92,000 MXN.

Ejecución: El consumo alcanza el 92.0%, sobrepasando el

umbral preventivo del 90%.

Resultado Esperado: Estado del semáforo en amarillo, bandera isOverrun = false, y generación de advertencia visual para restringir nuevas contrataciones no planificadas.

3. Escenario 1.3 — Sobrecosto Crítico (Semáforo Rojo):

Entrada: Presupuesto inicial de \$50,000 MXN; gastos registrados por \$55,000 MXN.

Ejecución: El gasto total rebasa el 100% pactado (110.0% consumido, balance deficitario de -\$5,000 MXN).

Resultado Esperado: Estado del semáforo en rojo, activación inmediata de la bandera de sobrecosto (isOverrun = true) y emisión de alerta crítica de déficit presupuestal.

4. Escenario 1.4 — Validación Defensiva (Caso Borde de Presupuesto en Cero):

Entrada: Evento sin presupuesto inicial asignado (\$0 MXN) que registra un gasto operativo imprevisto de \$1,500 MXN.

Ejecución: Algoritmo defensivo que previene divisiones por cero (NaN o Infinity).

Resultado Esperado: El balance refleja con precisión -\$1,500 MXN y activa el semáforo en rojo sin interrumpir la ejecución del sistema.

Caso de Prueba 2: Control de Aforo y Asignación de Mesas (CP-MES-02)

- Módulo / Componente: GuestService / TableService (Módulo de Invitados y Logística de Salón).
- Propósito: Comprobar que el algoritmo de distribución de salón respete estrictamente la capacidad física máxima de cada mesa, considerando tanto al invitado titular como a sus acompañantes confirmados (+1s).
- Precondiciones: Mesas del recinto dadas de alta con su capacidad nominal de sillas.
- Escenarios de Prueba Evaluados:

1. Escenario 2.1 — Asignación Válida con Acompañante:
Entrada: Mesa #4 con capacidad máxima de 10 personas; actualmente ocupada por 6 personas. Se asigna a un invitado titular que confirmó 1 acompañante adicional (+1).
Ejecución: El sistema calcula la ocupación total resultante (6 sentados + 1 titular + 1 acompañante = 8 personas).
Resultado Esperado: Asignación autorizada exitosamente (true), asociando al invitado a la mesa y actualizando el aforo ocupado a 8 comensales.
2. Escenario 2.2 — Rechazo por Sobrecupo:
Entrada: Mesa #2 con capacidad máxima de 10 personas; ocupada actualmente por 9 personas. Se intenta asignar a un invitado que confirmó 2 acompañantes (+2).
Ejecución: El sistema detecta que la demanda excede el aforo disponible ($9 + 1 + 2 = 12 > 10$).
Resultado Esperado: Asignación rechazada (false), denegación de la operación en base de datos y emisión del error de validación TableCapacityExceededError.

Caso de Prueba 3: Gestión de Menús, Alergias y Reporte de Catering (CP-CAT-03)

- Módulo / Componente: GuestService / CateringReportService (Módulo de Invitados y Gastronomía).
- Propósito: Validar la captura fidedigna de preferencias alimentarias y la consolidación de advertencias médicas críticas para prevenir intoxicaciones o emergencias en el banquete.
- Precondiciones: Invitados registrados con confirmación de asistencia activa (RSVP = confirmed).
- Escenarios de Prueba Evaluados:
 1. Escenario 3.1 — Registro y Sellado de Alergias Severas:
Entrada: Invitado confirma asistencia, selecciona menú "Vegetariano" y documenta alergia médica severa a

"Cacahuates / Frutos Secos (Riesgo de Choque Anafiláctico)".

Ejecución: La entidad de dominio valida y sella los datos, activando automáticamente el indicador de riesgo médico `hasCriticalAllergies = true`.

Resultado Esperado: Registro exitoso con etiquetado de alta prioridad médica vinculado permanentemente al número de mesa asignada al comensal.

2. Escenario 3.2 — Emisión del Reporte Oficial para Banquetero:

Entrada: Lista de 150 comensales confirmados con diversas opciones de platillo (regular, vegano, kosher, infantil) y 4 comensales con alergias médicas críticas.

Ejecución: Se ejecuta el método agregador `generateCateringDietaryReport(eventId)`.

Resultado Esperado: Generación de un reporte estructurado que detalla: (a) consolidado total de platillos por menú para compra de insumos, (b) conteo de charolas requeridas por mesa, y (c) sección de alertas rojas médicas destacando Comensal, Mesa y Alérgeno prohibido.

Caso de Prueba 4: Agenda Minuto a Minuto y Detección de Conflictos Horarios (CP-AGE-04)

- Módulo / Componente: AgendaService (Módulo de Itinerario Operativo).
- Propósito: Verificar que el algoritmo de cronograma detecte empalmes o solapamientos de horario entre proveedores o actividades que compiten por el mismo espacio físico o por el mismo responsable.
- Precondiciones: Evento activo con actividades programadas con hora de inicio y hora de fin $[t_{inicio}, t_{fin}]$.
- Escenarios de Prueba Evaluados:

1. Escenario 4.1 — Itinerario Secuencial Válido:

Entrada: Actividad A ("Vals de los Novios", 20:00 a 20:30, Pista Central) y Actividad B ("Apertura de Cena", 20:30 a 22:00, Salón Principal).

Ejecución: El comparador de intervalos evalúa que los rangos de tiempo sean disjuntos.

Resultado Esperado: Cero conflictos reportados (`conflicts.length = 0`), permitiendo guardar y publicar el itinerario oficial.

2. Escenario 4.2 — Detección Automática de Choque de Horarios:

Entrada: Actividad A ("Show de Mariachi", 21:00 a 22:00, Pista Central) y Actividad B ("Show de Baile Sorpresa", 21:30 a 22:30, Pista Central).

Ejecución: El algoritmo detecta solapamiento en el mismo espacio físico ($\max(t_{inicio,A}, t_{inicio,B}) < \min(t_{fin,A}, t_{fin,B})$).

Resultado Esperado: Identificación inmediata del conflicto (`conflicts.length = 1`), especificando los IDs de las actividades en colisión y la locación en disputa.

Caso de Prueba 5: Seguridad, Autenticación y Autorización RBAC (CP-SEC-05)

- Módulo / Componente: AuthService / RbacMiddleware (Módulo de Usuarios y Seguridad).
- Propósito: Garantizar el aislamiento estricto de información entre organizadores (*Multitenancy*) y el cumplimiento de restricciones operativas según el rol del usuario autenticado.
- Precondiciones: Cuentas creadas para los tres roles del sistema (admin, organizer, client).
- Escenarios de Prueba Evaluados:
 1. Escenario 5.1 — Autenticación y Emisión de Token JWT:

Entrada: Credenciales legítimas de un organizador (correo y contraseña válida).

Ejecución: Verificación de hash con Web Crypto API y firma criptográfica HMAC-SHA256 del token con expiración a 8 horas.

Resultado Esperado: Token JWT generado correctamente, conteniendo el identificador de usuario y el rol organizer.

2. Escenario 5.2 — Restricción de Privilegios para Rol Cliente:

Entrada: Petición HTTP POST hacia la ruta protegida de registro de gastos `/api/v1/events/{id}/expenses`, enviada con el token de un usuario con rol client.

Ejecución: El middleware de autorización intercepta la petición y valida la matriz de permisos RBAC.

Resultado Esperado: Acceso denegado con código HTTP 403 Forbidden, impidiendo que el cliente modifique presupuestos o contratos que corresponden exclusivamente al organizador.

3. Escenario 5.3 — Revocación de Sesión Inmediata (Logout via Cloudflare KV):

Entrada: Usuario cierra sesión y se envía su token JWT vigente al endpoint de invalidación.

Ejecución: El identificador único del token (jti) se inserta en Cloudflare KV con clave `blacklist:<jti>` y un tiempo de vida (TTL) idéntico al remanente del token.

Resultado Esperado: Cualquier petición posterior utilizando ese mismo token es rechazada automáticamente con código HTTP 401 Unauthorized.

Docker Hub y Estrategia de Contenedores

Para garantizar la reproducibilidad hermética, el aislamiento de dependencias y el cumplimiento de los estándares de la materia *Infraestructura para el Desarrollo Continuo*, los componentes de software de **PLANORA** se empaquetan como imágenes de contenedores compatibles con el estándar OCI (*Open Container Initiative*) y se publican automáticamente en el registro público de **Docker Hub**.

1. Repositorios Oficiales en Docker Hub

Se definen dos repositorios independientes bajo la cuenta del desarrollador:

- **Backend API (docker.io/<usuario>/planora-api):**
 - **Imagen Base:** node:20-alpine (distribución ultra ligera de Linux para minimizar superficie de vulnerabilidades).
 - **Estrategia:** Compilación multi-stage (*multi-etapa*) que separa el entorno de dependencias y pruebas del artefacto final de ejecución, logrando una imagen final optimizada de menos de 80 MB.
- **Frontend SPA (docker.io/<usuario>/planora-frontend):**
 - **Imagen Base:** nginx:alpine.
 - **Estrategia:** Compila los archivos estáticos de React 18 con Vite en una primera etapa y transfiere únicamente la carpeta dist al servidor Nginx ligero para servir la aplicación web.

2. Estrategia de Versionado y Etiquetas (*Tagging*)

Para mantener inmutabilidad, trazabilidad y control de versiones en cada entrega:

- **latest:** Apunta siempre a la versión más reciente, estable y probada proveniente de la rama principal (main).
- **<commit-sha> (ej. sha-7a8b9c0):** Etiqueta inmutable generada a partir del hash del commit de Git. Permite auditar exactamente qué versión de código fuente originó cada imagen desplegada.
- **v1.0.0 / SemVer:** Etiquetas semánticas asociadas a los *Releases* e hitos oficiales de entrega académica.

3. Pipeline de Automatización con GitHub Actions

El proceso de construcción y publicación no se realiza manualmente; está 100% automatizado dentro del pipeline de CI/CD:

1. **Pruebas Previas:** El trabajo de Docker solo se dispara si el linter, el typecheck y las pruebas unitarias con Vitest finalizan con éxito ($\geq 85\%$ de cobertura).

2. **Autenticación Segura:** El agente de GitHub Actions inicia sesión en Docker Hub utilizando credenciales cifradas almacenadas en los secretos del repositorio (DOCKER_USERNAME y DOCKER_TOKEN).
3. **Build & Push:** Se compilan en paralelo las imágenes del backend y frontend mediante *Docker Buildx* y se hace el push inmediato al registro público de Docker Hub con sus etiquetas correspondientes

Estrategia de Ramas Git

Se adopta un flujo **Git Flow Adaptado**:

- **Ramas Troncales:** main (código productivo 100% verificado) y develop (integración continua).
- **Ramas de Trabajo:** feature/<id>-<nombre> para desarrollo de requerimientos y fix/<id>-<nombre> para corrección de bugs.
- **Convención de Commits:** *Conventional Commits v1.0.0* (feat:, fix:, test:, docs:, refactor:).
- **Integración:** Fusiones obligatorias mediante *Pull Request* condicionados a la aprobación de los *Status Checks* automatizados de GitHub Actions.

GitHub Project Board y plan de trabajo



Relación de Milestones

- M1: Setup del Monorepo y Herramientas Base.
- M2: Base de Datos D1 y Esquemas Drizzle.
- M3: Autenticación JWT y Roles RBAC.
- M4: Módulo Core (Eventos, Clientes, Invitados y Mesas).
- M5: Motor Financiero, Gastronómico y Operativo.
- M6: Pruebas Unitarias Automatizadas en Vitest.
- M7: Containerización con Docker Multi-stage.
- M8: Pipeline CI/CD en GitHub Actions y Docker Hub.

M9: Despliegue en Producción a Cloudflare Workers y Pages.

M10: Documentación Final y Preparación de la Defensa.

#	Issue	Milestone	Prioridad	Dependencias
#01	Setup del Monorepo, TypeScript, ESLint y Wrangler	M1	Crítica	Ninguna
#02	Esquema Relacional D1 y Migraciones con Drizzle ORM	M2	Crítica	#01
#03	Módulo de Autenticación JWT y Roles RBAC (admin, organizer, client)	M3	Alta	#02
#04	Revocación de Tokens (Logout) y Rate Limiting con Cloudflare KV	M3	Media	#03
#05	CRUD de Clientes y Aislamiento Multitenant por Organizador	M4	Alta	#03
#06	Módulo de Eventos y Máquina de Estados del Ciclo de Vida	M4	Crítica	#05
#07	Módulo de Invitados, Confirmación RSVP y Aforo de Mesas	M4	Alta	#06
#08	Directorio de Proveedores y Catálogo de Servicios Base	M5	Media	#03
#09	Contratación de Servicios (event_services) y Almacenamiento en Cloudflare R2	M5	Alta	#06, #08
#10	Motor de Presupuesto, Gastos y Semáforo de Sobrecosto	M5	Crítica	#06, #09
#11	Tablero de Tareas y Cálculo Dinámico de Progreso Porcentual	M5	Alta	#06
#12	Cronograma Minuto a Minuto con Detección de Conflictos Horarios	M5	Alta	#06
#13	Endpoints del Dashboard Ejecutivo de Métricas y KPIs	M5	Media	#10, #11, #12
#14	Frontend: Setup de React 18, Vite, Tailwind CSS y Enrutamiento Protegido	M4	Alta	#03

#	Issue	Milestone	Prioridad	Dependencias
#15	Frontend: Vistas de Dashboard, Detalle de Evento y Presupuesto	M5	Alta	#13, #14
#16	Suite Integral de Pruebas Unitarias con Vitest ($\geq$ 85% de cobertura del dominio)	M6	Crítica	#10, #11, #12
#17	Dockerización del Backend con Dockerfile Multi-Stage y Healthcheck	M7	Alta	#01, #16
#18	Dockerización del Frontend con Nginx Alpine y Docker Compose Local	M7	Alta	#15, #17
#19	Configuración de Pipeline CI en GitHub Actions (Lint, Types, Tests)	M8	Crítica	#16
#20	Automatización de Construcción y Push a Docker Hub	M8	Crítica	#17, #18, #19
#21	Automatización de Despliegue a Cloudflare Workers y Pages con Wrangler	M9	Crítica	#20
#22	Documentación Técnica de Arquitectura y Especificación en Wiki	M10	Alta	Todas
#23	Elaboración de Guion y Estructura de Diapositivas de Presentación	M10	Alta	#22